

Questions Abstraction and Encapsulation

1. Create a `BankAccount` class that stores:

- account number
- balance (should not be directly modifiable)

You must:

1. Make the balance attribute inaccessible from outside.
2. Provide functions to deposit/withdraw that validate the amount.
3. Prevent withdrawal if balance becomes negative.
4. Show what happens if someone tries to modify balance directly and why encapsulation prevents it.

2. Design a `Student` class where marks:

- should always be between **0 and 100**
 - should never be set directly
- Enable updating marks **only through a controlled method** that performs range checks.

Demonstrate:

- trying to assign marks manually
- why encapsulation protects invalid states

3. Create a `SecureFile` class that:

- stores content privately
- provides a method `read(password)`
- refuses access if the password is incorrect
- logs an "Unauthorized attempt" internally (cannot be accessed from outside)

4. Design an `Employee` class where:

- salary is hidden
- outsiders cannot read salary directly
- use getter method that logs each access attempt
- provide a method to update salary but only if the new salary is higher (prevent accidental downgrade)

5. Create a `Product` class where:

- price cannot be negative
- discount cannot exceed 70%

- internal final price calculation should not be directly exposed
Provide only one public method `get_final_price()`.

6. Create a `Character` class with:

- `private _health`
- methods to `damage(points)` and `heal(points)`
- health cannot drop below 0 or exceed max limit
- expose only current health through a read-only getter

7. Create:

- An `Engine` class with private state like temperature
- A `Car` class that uses an `Engine` but should:
 - Not allow users to manipulate engine temperature
 - Only expose methods like `start_car()` or `cool_engine()`

Demonstrate why giving direct engine access is dangerous.

8. Create a `ShoppingCart` class where:

- items are stored privately
- users cannot directly modify item list
- only add/remove methods are allowed
- provide a method to get a safe copy of the cart items (not direct reference to internal list)

9. Implement a class incorrectly first:

- Attendance stored in a list
- Exposed directly so any outside code can modify it

Then redesign properly:

- Make attendance private
- Provide controlled methods for marking attendance only

Explain the difference.

10. Create a class using `@property` and `@setter` for a private attribute.
Then:

1. Show correct usage
2. Show how forgetting to use underscore prefix breaks encapsulation
3. Show what happens if you implement a setter without validation

Focus: Python-specific encapsulation pitfalls, misuse of properties.

11. Using `abc` module:

- Create an abstract class `Shape` with `area()`, `perimeter()`
- Implement `Circle`, `Rectangle`, `Triangle`

Demonstrate:

- why base class should NOT contain calculation logic
- what happens if a subclass fails to implement one of the methods

12. Design an abstract class `PaymentGateway` with:

- `authenticate()`
- `pay(amount)`
- `refund(amount)`

Implement subclasses:

- `UPIPayment`
- `CardPayment`
- `NetBankingPayment`

Show how abstraction helps your main program call payment methods without caring about the payment type.

13. Create:

- Abstract class `VehicleControl` with methods `accelerate()`, `brake()`, `steer()`
- Implement `CarControl`, `BikeControl`, `TruckControl`

Demonstrate calling each through a single interface.

14. Create an abstract class `DatabaseDriver` with:

- `connect()`
- `execute(query)`
- `close()`

Implement concrete drivers:

- `MySQLDriver`
- `PostgresDriver`
- `SQLiteDriver`

Show how abstraction helps switch databases without rewriting main code.

15. Design a class `ReportGenerator` (abstract) with:

- `load_data()`
- `process()`
- `export()`

Implement:

- `PDFReport`
- `ExcelReport`

Demonstrate how abstraction enforces a multi-step structure.

16. Create an abstract class `RobotCommand` with:

- `execute()`
- `undo()`

Implement:

- `PickCommand`
- `PlaceCommand`
- `MoveCommand`

Demonstrate how abstraction cleanly represents commands without revealing details.

17. Create an abstract class `MLModel` with:

- `train(data)`
- `predict(x)`
- `evaluate(test_set)`

Implement models:

- `LinearRegressionModel` - some different logic
- `DecisionTreeModel` - some logic

Show how a generic training loop works for any model without caring about details.

18. Design a system **without abstraction** first:

- Write separate functions for `EmailSender`, `SMSSender`, `PushSender`
Show how the main program becomes a mess with constant `if/else`.

Then:

- Redesign using an abstract base class `Notifier`.

19. Create an abstract `MediaPlayer` with:

- `load()`
- `play()`
- `stop()`

Implement:

- MP3Player
- WAVPlayer
- AACPlayer

Demonstrate calling each via a unified interface.

20. Design:

- Abstract base class `Sensor` with functions `read_value()` and `calibrate()`
- Subclasses: `TemperatureSensor`, `PressureSensor`, `HumiditySensor`

Encapsulate:

- internal raw sensor readings
 - calibration factor
- Hide all raw operations and allow only a public, clean `get_reading()` method.